Важно!
Чтобы продолжать учиться и учить

Подробнее

Цикл while. Учимся крутиться как белка в колесе

- Условие входа в цикл
- Бесконечный цикл
- Сокращённые операторы
- Моржовый оператор
- Множественное присваивание
- Поиск максимума
- Умный выбор

Представьте, что вы почти древнегреческий Сизиф. Ваша задача — написать программу, которая будет катить камень в гору до выполнения какого-нибудь условия, например пока Зевс не скажет «стоп». В жизни и в программировании часто возникает необходимость повторять действия, пока не случится какое-нибудь желаемое (или, наоборот, нежелательное) событие.

И сегодня мы узнаем, какая программа работает у котика, который мяукает до тех пор, пока его не покормят. Поехали!

🔄 Пока ложь не разлучит нас

Python (как и любой другой язык программирования) поддерживает циклы — возможность выполнять какой-нибудь участок кода многократно. Операторов циклов в Python больше одного — целых два. И сегодня мы посмотрим на `while` («пока»), который называют *циклом с неизвестным количеством повторений*. Блок кода внутри `while` выполняется до тех пор, пока истинно условие в заголовке цикла.

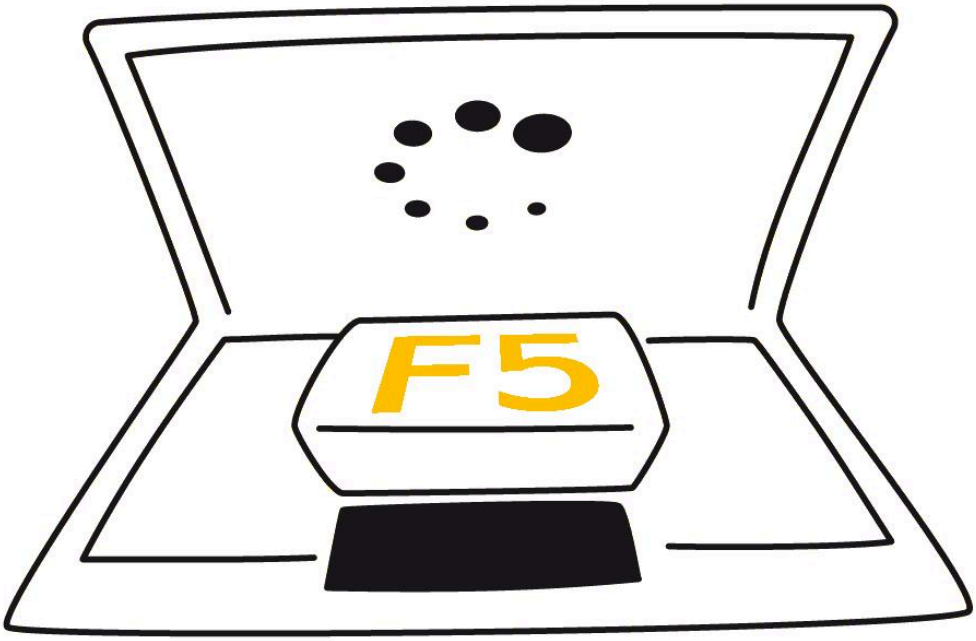
```
while <условие>:  # Заголовок.
    <блок кода>  # Тело цикла. Не забываем про 4 пробела.
```

Оператор `while` («пока») тоже проверяет условие и тоже в случае его истинности выполняет следующий блок кода (**тело цикла**). Однако после выполнения этого блока кода выполняется не то, что идёт после него, а **снова проверяется условие**, записанное после `while`. И так далее, пока условие в заголовке истинно.

Условие цикла записывается как и для `if`:

- с помощью операций отношения (>, >=, <, <=, !=, ==);
- с использованием логических операций `not`, `and`, `or`;
- с помощью переменных типа `bool` (или неявного приведения переменных других типов к `bool`).

Один оборот цикла (выполнение всего тела цикла) называют **итерацией**.



А это мы проверяем появление результатов важного экзамена каждую минуту, пока они не появились:

```
got_results = False  # Выполнится перед началом цикла.

while not got_results:  # not от False -> True, пока нет результатов — не
    print("🔄 Проверяю почту...")
    print("Результаты пришли?")
    if input() == 'да':
        got_results = True  # Ура!

print("📩 Письмо получено!")  # Выполнится после завершения цикла.
```

Если условие изначально ложно, то цикл не выполнится вообще ни разу!

Ещё пример. Посмотрим на программу, которая иллюстрирует длительную поездку с младшим братом. В ней цикл будет выполняться до тех пор, пока пользователь не введёт «да» или «приехали»:

```
question = 'Мы уже приехали?'
print(question)
answer = input()
while answer != 'да' and answer != 'приехали':
    print(question)
    answer = input()
print('Отличная поездка!')
```

Разберём, как работает эта программа, чуть подробнее.

Сначала выполняются первые строчки: мы запоминаем в переменной `question` вопрос, чтобы не дублировать его потом дважды (так проще исправлять опечатки и менять его на другой), печатаем его, считываем первый ответ пользователя.

Предположим, он ввёл «Пока нет». Оно записано в переменной `answer`.

Выполняется строчка: `while answer != 'да' and answer != 'приехали':` — здесь проверяется, выполнено ли условие. Поскольку мы предположили, что `answer` в этот момент равно «Пока нет», тогда условие выполнено, поэтому дальше выполняется блок, записанный с отступом, — тело цикла.

Мы снова выводим вопрос и считываем пользовательский в переменную `answer`. Пусть пользователь ввёл «Всё ещё нет».

Когда выполнение программы доходит до конца тела цикла, происходит возврат к заголовку цикла и повторная проверка условия. Поскольку мы так и не получили желаемый ответ, снова выполняется тело цикла.

И так до тех пор, пока мы не приедем.

♾️ Бесконечный цикл: в вечном ожидании звонка

Мы сказали, что если условие изначально ложно, то цикл не выполнится ни разу, но что, если оно будет всегда истинно? Добро пожаловать в бесконечный цикл.

```
while True:
```

Всегда проверяйте, есть ли в цикле условие для выхода!

> Если вы случайно или специально сделали бесконечный цикл, то не надо перезагружать компьютер, можно его остановить, если в консоли набрать `ctrl + c`.

Можно предположить, что бесконечные циклы — это какая-то программистская шутка, чтобы разыгрывать новичков, но у них есть и практическое применение:
вы обязательно столкнётесь с ними, когда будете писать игры, веб-приложения или ботов для различных платформ.

➕ Сокращённые операторы. Лень на страже краткости (или наоборот)

Надеемся, что вы не забыли, что в операторе присваивания одно и то же имя переменной может стоять и справа (в составе какого-то выражения), и слева. В этом случае сначала вычисляется правая часть со старым значением переменной, после чего результат становится новым значением этой переменной.

```
counter = 0  # Было ноль.
counter = counter + 1  # А стало на один больше.
```

Это не уравнение, у которого надо привести подобные 😈

Но поскольку программисты стремятся к лаконичности, а ещё и довольно ленивы, то, конечно же, придумали, как записать такую конструкцию короче. Встречайте: сокращённая форма записи оператора присваивания (для примера выше):

```
counter = 0  # Было ноль.
counter += 1  # А стало на один больше, да и ещё как красиво написано!
```

Такая форма записи есть у любого оператора, потому что Python не терпит никакой дискриминации:

- += (добавить)
- -= (вычесть)
- *= (умножить)
- /= (разделить)
- //= (целочисленно разделить)
- **= (возвести в степень)
- %= (найти остаток от деления)

🎑 Сейчас всё посчитаем

Очень часто такую сокращённую форму используют в циклах `while` для изменения переменной, которая участвует в проверке условия.

```
cookie_count = 10
while cookie_count > 0:
```

```
    cookie_count -= 1  # ← Сокращённый оператор как диета для кода.
print("Теперь я без печенек.")
```

Не забываем две очень важные вещи:

- задать **начальное значение** переменной;
- **изменять её в цикле**, чтобы когда-нибудь из него выйти.

⛔ Сигнал остановки — наш спасительный стоп-кран

Рассмотрим такую задачу: пользователь вводит какие-нибудь значения, причем мы заранее не знаем, сколько их будет. Пусть это будут числа — возраст котиков. И пользователь решил посчитать суммарный возраст всех своих домашних животных. В этом случае надо подумать, как пользователь может нам сообщить о том, что ввод закончился.

Для сигнала остановки выбирают значение, которое **точно не может встретиться в валидном** пользовательском вводе.

Так как мы считаем возраст, условимся, что отрицательным он быть не может, поэтому ввод «–1» — сигнал остановки.

Поскольку требуется повторить нечто (например, ввод возраста очередного котика) неизвестное количество раз, потребуется цикл `while`. Нам понадобится как минимум две переменные: `current_cat_age` для возраста котика, обрабатываемого в данный момент, и `total_age` — для суммарного количества лет.

Называйте переменные осмысленно, пожалуйста, пожалуйста! Это очень облегчит жизнь программисту, который будет читать ваш код позже. Скорее всего, это будете вы сами некоторое время спустя.

Если бы мы знали точно, что пользователю надо сосчитать суммарный возраст ровно трёх котиков, то можно было бы написать что-то вот такое и даже обойтись без цикла:

```
total_age = 0
current_cat_age = int(input())
total_age += current_cat_age
current_cat_age = int(input())
```

```
total_age += current_cat_age
print('Суммарный возраст котиков', total_age)
```

Если вы написали какой-то кусочек кода дважды (или, о ужас, трижды), значит, вы что-то делаете не так.

Очевидно, что можно организовать цикл со счётчиком, который внутри цикла будет считать каждую итерацию. А условием выхода обозначим выполнение нужного количества итераций:

```
count = 0
total_age = 0
while count < 3:
    current_cat_age = int(input())
    total_age += current_cat_age
    count = count + 1
print('Суммарный возраст котиков:', total_age)
```

Однако у нас в задаче количество котиков неизвестно, поэтому понадобится цикл до ввода сигнала остановки (–1):

```
total_age = 0
print('Посчитаем суммарный возраст всех ваших котиков.')
print('Вводите возраст; для остановки введите -1.')
current_cat_age = int(input())
while current_cat_age != -1:
    total_age += current_cat_age
    current_cat_age = int(input())
print('Суммарный возраст котиков:', total_age)
```

Моржовый оператор (:=): «Улыбочку»

Оператор := (похож на моржовые клыки) позволяет присвоить значение переменной прямо в условии. Это упрощает код, объединяя присваивание и проверку.

Было:

```
print("Введи 'quit' для выхода")
```

```
    print(f"Ты написал: {text}")
    text = input()
print("Выход!")
```

Стало:

```
print("Введи 'quit' для выхода")
while (text := input()) != "quit":
    print(f"Ты написал: {text}")
print("Выход!")
```

Скобки нужны для того, чтобы повысить приоритет присваивания (а он самый маленький). Благодаря скобкам сначала выполнится `input()` и результат пользовательского ввода запишется в переменную `text`, и только потом пользовательский ввод сравнится со строкой «quit». Без скобок приоритета пользовательский ввод сначала сравнился бы со строкой, а потом результат этого сравнения (`True` или `False`) записался бы в переменную `text`.

Ещё пример:

```
# Без моржа:
print("Введите пароль:")
text = input()
while len(text) < 8:
    print("Пароль слишком короткий!")
    print("Введите пароль:")
    text = input()

# С моржом (и небольшим жульничеством с параметром функции input()):
while len(text := input("Введите пароль:\n")) < 8:
    print("Пароль слишком короткий!")
```

⇄ Множественное присваивание. Оптом дешевле

Если нам захочется (а нам будет часто хотеться) поменять местами значения двух переменных, то естественное желание — сделать такой обмен через третью переменную:

```
# Для обмена подкинем что-то в воздух,
air = right_hand
# переложим,
right_hand = left_hand
# поймаем:
left_hand = air
```

В Python благодаря магии множественного присваивания мы можем обойтись и без третьей переменной (Что? Да!):

```
right_hand = 'apple'
left_hand = 'orange'

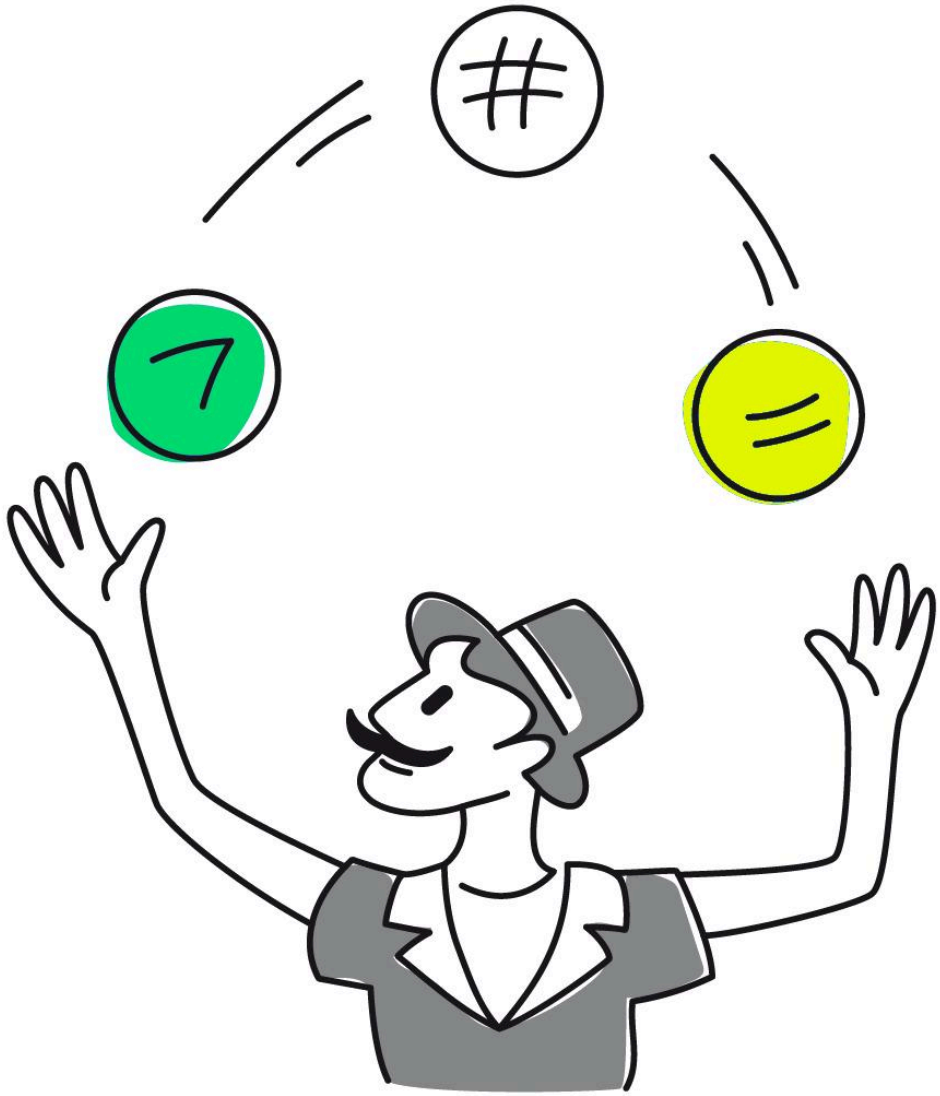
right_hand, left_hand = left_hand, right_hand  # Переменные обменялись зн
```

А ещё такой прием можно использовать, чтобы присвоить значения нескольким переменным в одной строке:

```
cat_name, dog_name = "Барсик", "Шарик"
```

Можно жонглировать любым количеством переменных:

```
# Обмен значениями трёх переменных за один присест:
x, y, z = "огонь", "вода", "земля"
x, y, z = z, x, y  # Теперь x = "земля", y = "огонь", z = "вода".
```



Не забываем пробелы после запятой, чтобы не прогневать PEP8.

С этими знаниями давайте напишем программу, где мы будем каждый раунд перепрятывать «шарик» из одной переменной в другую.

```
print("Сейчас шарик в правом стаканчике!")
left_cup, right_cup = "", "шарик"
# Перед каждым запуском тихонечко, чтобы никто не видел, меняем значения
swap_count = 9  # Ох, тут бы нам пригодилась возможность генерировать слу
current_round = 0
print("Кручу-верчу, запутать хочу")
```

```
print("В каком стаканчике шарик? 1 - в левом, 2 - в правом")
answer = int(input())
if answer == 1 and left_cup == "шарик" or answer == 2 and right_cup == "ш
    print("Вы победили!")
else:
    print("Получится в другой раз")
```

И даже это не все возможности множественного присваивания (а точнее, механизма запаковки/распаковки, который за ним стоит). Глубже мы копнём чуть позднее.

⛰️ Поиск максимума. Всегда хочется большего

Или меньшего, если ищем минимум.

Конечно, максимум и минимум — не единственное, что ищут программисты, но приёмы для остальных статистических алгоритмов (среднего, медианы, моды и т. д.) довольно похожи.

В таких случаях всегда заводится дополнительная переменная для хранения промежуточного результата. Для минимума — заведомо большое значение (чтобы любое введённое значение было меньше), для максимума — наоборот, заведомо маленькое.

Также хорошим приёмом может быть использование в качестве минимума или максимума просто первого введённого элемента.

В теле цикла каждый подходящий элемент данных обрабатывается операторами по принципу:

- если текущий элемент больше максимума, меняем максимум;
- если текущий элемент меньше минимума, заменяем минимум.

Конечно, не оставим вас без примера. Допустим, мы решили навести порядок в книжном шкафу, а заодно посмотреть, какая у нас самая толстая и самая тонкая книга. Мы последовательно будем вводить количество страниц в каждой следующей книге. Когда переберём все — введём отрицательное число или ноль (потому что в любой уважающей себя книге больше нуля страниц).

```
while (n := int(input())) > 0:  # Сразу вводим, записываем в переменную и
    # Сравниваем с помощью if.
    if n > biggest_book:
        biggest_book = n
    # Или выбираем нужное значение с помощью min|max.
    smallest_book = min(smallest_book, n)
# Сюда попадём, когда введём 0 или отрицательное число.
print(biggest_book)
print(smallest_book)
```

Запускаем, начинаем вводить количество страниц, например 148. 148 > 0 — условие цикла выполняется, и мы переходим к операции сравнения. На данном шаге 148 > 0,

- `biggest_book = 148`
- `smallest_book = 148`

Снова считываем число.

Предположим, теперь введено 120. 120 > 0 — продолжаем работать в цикле. 120 > 148 — условие для максимума не выполнится, а вот минимум заменим:

- `biggest_book = 148`
- `smallest_book = 120`

В этот раз введем 486. Мы заходим в цикл 486 > 148, производим замену максимума, минимум не изменится.

- `biggest_book = 486`
- `smallest_book = 120`

Продолжаем ввод. И так далее до тех пор, пока не будет введено отрицательное число или 0.

💡 Умный выбор: `match/case` внутри

Как мы уже говорили ранее (и ещё парочку раз напомним дальше), **в Python в любом блочном операторе может быть другой блочный оператор**. Поэтому никто

```python
health = 100
run_away = False

while health > 0 and not run_away:  # Пока есть здоровье и не убежали.
    print("Ваш ход (атака/защита/бег):")
    match input():  # Нам даже не надо хранить ввод — сразу выбираем.
        case "атака":
            print("⚔️ Атакую врага!")
            health -= 10  # Тратим силы.
        case "защита":
            print("🛡️ Защищаюсь!")
            health += 5  # Восстанавливаемся.
        case "бег":
            print("🏃 Бегу с поля боя!")
            run_away = True
        case _:
            print("🤷 Неизвестная команда.")

    print(f"❤️ Здоровье: {health}")

if health <= 0:
    print("💀 Игра окончена.")
else:
    print("🏆 Победа!")
```

🌟 Итог: цикл жизни нам стал чуть понятнее

Цикл `while` — это вечный двигатель вашей программы. Мы запустили этот двигатель, а также:

- начали экономить символы с сокращенной записью присваивания;
- научились жонглировать значениями переменных;
- смогли создать бесконечный цикл и прервать бесконечность с помощью `ctrl+c`;
- познакомились с моржовым оператором без похода в зоопарк;
- а ещё рассмотрели много типовых примеров использования цикла while.